1 重编程 ECU 需求

1.1 不可重编程 ECU 的通用需求

为确保重编程的成功执行，连接在 CAN 网络上的不可重编程 ECU 需要满足如下要求：

支持诊断会话控制-切换到扩展会话模式（即$10 服务中的$03 子功能），通 过此服务，ECU 可进入扩展会话模式。

支持诊断仪在线服务（即$3E 服务$00/$80 子功能），通过此服务，可使 ECU 维持在非默认会话模式下。

支持通信控制-禁止/使能发送和接收（即$28 服务中的$00/$03 子功能）的
诊断服务，通过此服务，可以降低编程过程中总线的负载；
支持控制 DTC 设置（$85）诊断服务，通过此服务，确保当某个 ECU 处于编程状态时，其他 ECU 不会设置 DTC；

不支持重编程相关的诊断服务，如：诊断会话控制-切换到编程模式（$10 $02）、请求下载（$34）、传输数据（$36）、请求传输退出（$37）。

不可重编程 ECU 需要支持第 3.3.1 节中规定的诊断服务，除了“例程控制- 检查预编程条件”以及“通过标识符读数据”两个服务。

1.2 重编程 ECU 的需求

1.2.1 概述

所有支持应用软件及应用数据（包括网络配置数据和标定数据）重编程的 ECU，应当包含 Bootloader 软件。在正常运行过程中，执行的是应用软件和应
用数据。仅当应用软件或应用数据无效时，或者要求对此进行升级的时候，Bootloader 软件才被激活。

应用软件和应用数据可以同时编程或者相互独立编程，不允许重编程时更新
Bootloader 软件。

1.2.2 硬件需求

任何可重编程的并且正确安装的 ECU 可通过诊断接口进行编程。在重编程
过程中不需要拆除 ECU。 可重编程的 ECU 必须提供足够的存储器来保证下载，提供充足的缓冲空间来满足重编程定时要求。用于编程的网络层缓冲区所占的字节数应该得到启源认可。

1.2.3 软件需求

Bootloader 软件必须存储在被保护的存储器中，从而确保即使存在潜在错误，
ECU 始终是可重编程的。

1.2.4 安全需求

为确保下载的安全性，ECU 应该实现一些安全需求，避免下面几种情况：

1) 来自非法源的下载动作；
2) 当前重编程条件不满足；
3) 下载错误的应用软件或者应用数据到 ECU；
4) 软件之间不兼容。

1.2.4.1 安全访问

有可重编程的 ECU 应该支持种子和密钥的安全特性，并且可以通过安全访问服务（$27）进入访问，从而保护 ECU 免遭未授权的编程动作影响。

安全等级 $09/$0A 被用于编程会话模式，详细的要求请参考启源提供的安全算法文档。

1.2.4.2 预编程条件

ECU 应该确保重编程的执行是处于安全状态。如果预编程条件不满足（如车辆或发动机正在运行等），那么重编程请求将被拒绝。

通过一个“检查预编程条件”例程控制来激活 ECU 预编程条件的检验，详 细信息请参考第 3 章。

1.2.4.3 完整性验证

ECU 需要检查下载到存储器中的数据的完整性。当一个逻辑块下载后，将使用 CRC32 算法验证当前逻辑块的所有数据字节是否被正确传输和写入。通过 一个“检查编程完整性”例程控制来激活 ECU 完整性验证。当 ECU 接收到此服务请求时，Bootloader 将计算下载数据字节的 CRC32 值，并将计算结果与诊断仪请求报文中发送的校验值进行比较。

CRC32 的多项式为 0x04C11DB7，并使用如下的多项式生成：

$G(x) = x_{32} + x_{26} + x_{23} + x_{22} + x_{16} + x_{12} + x_{11} + x_{10} + x_8 + x_7 + x_5 + x_4 + x_2 + x + 1$

IEEE 802.3 规定 CRC-32 使用的初始值为 0xFFFFFFFF，校验结果需和 0xFFFFFFFF 按位进行异或计算。

1.2.4.4 依赖性检查

不兼容的软件不能配合使用，如果配合使用可能会使功能异常或产生致命性
错误。因此，ECU 应该通过验证软件兼容性来检查重编程依赖性，包括应用软
件与 Bootloader 软件、应用数据与应用软件等。依赖性检查机制由 ECU 供应商制定。

当 ECU 收到例程控制—“检查编程依赖性”诊断服务的请求时，ECU 将执行依赖性检查。详细的定义请参考第 3 章。

1.2.4.5 软件有效性验证

ECU 内部定义一个标志位，用于标识应用软件是否有效。如果重编程完整
性检查和重编程依赖性检查都正确，ECU 将设置应用软件的标志位为有效。只有标志位为有效时，应用软件才可以运行（详见第 2.2.8 节）。

1.2.4.6 内存驱动下载

内存驱动是一种执行初始化、擦除或写入内存功能的硬件从属软件。ECU 内存的内容必须是受保护的，防止意外的擦除或重写操作。因此，ECU 需要采用软件互锁机制，即用于编程的内存驱动并不是存储在 ECU 的永久性存储器中，而是在下载过程中下载到 ECU 的 RAM 中。下载完成后，驱动代码将从 ECU RAM 缓冲区彻底移除。

具体的内存驱动下载过程，详见第 3 章。

1.2.4.7 故障容错

Bootloader 软件将被存储在一个受保护的永久性存储器中，这样可以避免被
意外擦除，即使没有应用软件或应用数据，ECU 依然可以支持重编程。 ECU 在重编程过程中，发生以下情况时，应该可以重编程：

- 从低/高电压状态恢复；
- CAN 通信错误和超时恢复后；
- 重启；
- 擦除了部分 Flash 内存。

1.2.5 源文件格式要求

发布源文件格式是 Motorola 格式（*.s19）。

注意：s19 格式源文件如需支持分段（连续的地址为一段）刷写，则 ECU 应
支持任意分段的单独擦除。

1.2.6 通信需求

1.2.6.1 数据链路层参数

CAN ID 的分配由启源确定，而且应该与增强型诊断中用的 CAN ID 一致。

1.2.6.2 网络层参数

内存下载的网络层实现具体细节参考第 1.2.3 章。

1.2.6.3 诊断层参数

内存下载的诊断层实现具体细节参考第 1.2.4 章。

1.2.7 诊断服务需求

为了满足存储器编程的执行需求，第 3.4 节定义了应用软件和 Bootloader 软件支持的诊断服务的最小设置。

1.2.8 ECU 启动时序

在上电/复位后，ECU 首先执行 Bootloader 代码。Bootloader 首先执行一些基本的初始化，然后检查外部重编程请求标志位是否为有效，如果重编程

请求标志位为有效，即使应用程序是有效的，Bootloader 也会继续运行。如果当前没有重编程请求，即重编程请求标志位为无效，则检查应用软件的状态：如果应用软件是有效的，则应用软件代码将被执行；如果应用软件是无效的，则继续执行 Bootloader 代码。

1.3 网关需求

如果诊断报文需要通过网关转发，需要遵循下面规定：

- 地址为网关本身的物理寻址的诊断请求报文，则此诊断请求报文不需要被路由；
- 地址为非网关本身的物理寻址的诊断请求报文，则此诊断请求报文需要被路由到相应的网段；
- 地址为功能寻址的诊断请求报文，则网关必须将其路由到其连接的各个网段。

如果对网关 ECU 本身进行编程，则与非网关 ECU 不同的是：网关 ECU 的 Bootloader 必须在其连接的网段上路由或者发出诊断仪在线报文（$3E）。

2 重编程流程

2.1 概述

本章定义了将一个或多个应用软件或应用数据下载到 ECU 内存中的重编程流程。

2.2 Bootloader 启动时序

在应用模式下，使用了两种不同的诊断会话模式：默认会话模式和扩展会话模式。

在 Bootloader 模式下，使用了三种不同的诊断会话模式：默认会话模式，扩展会话模式和编程会话模式。

从诊断会话的角度来说，要进入编程会话模式，必须先通过扩展会话模式。

这意味着 ECU 不支持直接从默认会话直接跳转到编程会话模式。同样地，ECU

也不支持从编程会话直接跳转到扩展会话模式。

如果 ECU 在正确的条件下收到“$10 $02”，ECU 将外部重编程请求标志位置为有效，并执行 ECU 重启。

如果 ECU 在正确的条件下收到“$10 $02”，ECU 将外部重编程请求标志位置为有效，并执行 ECU 重启。

上电/复位后，ECU 首先执行 Bootloader 引导代码，然后检查外部重编程请求标志位：

- 如果外部重编程请求标志位为有效，那么即使应用程序是有效的，Bootloader 也会继续进一步执行，在此情况下，ECU 直接进入编程会话模式。
- 如果外部重编程请求标志位为无效，则继续检查应用软件的标志位状态：
- 如果应用软件是有效的，则启动应用模式；
- 如果应用软件无效，ECU 停留在 Bootloader 模式下的默认会话模式。

在 Bootloader 模式下，诊断会话转换规则与应用模式下相同。

在 Bootloader 模式下，有以下几种方式，会导致 ECU 重启：

- 无论当前处于何种会话模式，“$11 $01”都会导致 ECU 重启；
- 在扩展会话模式或编程会话模式下，S3 定时器超时会导致 ECU 重启；
- 在编程会话模式下，“$10 $01”会导致 ECU 重启。

2.3 重编程时序

重编程时序分为三个编程步骤：

- 预编程步骤：编程前的 CAN 网络准备；
- 主编程步骤：下载应用软件或应用数据；
- 后编程步骤：重同步 CAN 网络。

2.3.1 预编程步骤

预编程步骤用来为要下载的 ECU 做重编程前的 CAN 网络准备。此步骤也
包含了提高下载速度的准备。此步骤的请求报文能采用的是物理寻址，或功能寻
址。

（a）诊断会话控制 $10 $03：为了禁止 ECU 间的正常通信和控制 DTC 设置，预编程需要启动非默认会话模式。通过使用会话类型为扩展会话模式的诊断
会话控制（$10）服务来完成。此请求使用一个单帧请求报文，通过功能寻址发
送给所有的 ECU。完成扩展会话模式切换后，主刷节点应持续周期性（S3client
内）发送诊断仪在线服务请求（$3E $80），以维持所有 ECU 的非默认会话模
式。诊断仪在线服务请求使用一个单帧请求报文，通过功能寻址发送给所有 ECU。

（b）例程控制“检查预编程条件”$31 $01 $02 $03：通过此例程来检查 ECU 编程条件，从而确保系统安全，预编程检验条件由 ECU 决定，如果有任何不安全的因素，ECU 应该拒绝编程。

注意：如果 ECU 在未收到“检查预编程条件”例程（$31 $01 $02 $03）的情况下，收到“$10 $02”请求，ECU 应该拒绝进入 Bootloader 模式，并且
发送否定响应。

（c）控制 DTC 设置 $85 $02：诊断仪通过 DTC 设置类型设为“关闭”的控制 DTC 设置服务请求。此请求使用一个单帧请求报文，通过功能寻址发送给
所有的 ECU。

（d）通信控制 $28 $03 $03：诊断仪通过通信控制（$28）服务请求，禁止非诊断报文的发送和接收。请求中的控制类型参数置为“disable the transmission and the reception”，通信类型置为“application and network

management messages”。此请求使用一个单帧请求报文，通过功能寻址发送给所有的 ECU。
（e）读取数据 $22 $xx $yy：在禁止正常通信后，读取被编程的 ECU 的状态（如：编程的应用软件和数据）。从被编程的 ECU 读取服务器标识数据，标识如应用软件标识、应用数据标识、Bootloader 软件标识。读取数据服务为可选服务，读取的内容由 ECU 供应商定义。

2.3.2 主编程步骤

在预编程步骤之后，是主编程步骤。主编程时序是单个 ECU 编程事件的应
用，因此所有服务的请求都使用物理寻址。
（a）诊断会话控制 $10 $02：在收到一个会话类型为物理寻址的编程会话的诊断会话控制（$10） 服务后，ECU 开始编程条件检查。编程条件满足后，ECU 将分配编程须有的所有必须的资源，ECU 先发送肯定响应再执行跳转到编程模式动作；
（b）安全访问 $27 $09/$0A：编程事件必须通过安全访问。安全访问（$27）服务在排放相关和安全系统中是强制的。其它系统不要求使用该服务。下载前， 通过安全访问过程是强制的，确保只有合法的诊断仪能对 ECU 进行下载操作。
（c）驱动下载 $34，$36，$37，$31：当 ECU 的非易失性存储单元中没有存储内存驱动时，将执行内存驱动的下载。下载应该按照如下时序来进行：请求下载、传输数据、请求传输退出。下载完所有字节后，用“检查编程完整性”例程（$31 $01 $02 $02）来检查所有的字节都正确传输。
（d）写入数据$2E $F1 $5A：在擦除内存例程之前，将“指纹”写到 ECU 内存中是强制的。“指纹”标识了是哪个诊断仪对 ECU 内存做了修改。我们使用 $F1 $5A 数据标识符而不是引导软件指纹、应用软件指纹、应用数据指纹这些数据标识符。每个逻辑块（除了驱动）下载前，诊断仪将首先写“指纹”，在下载完逻辑块后，ECU 即知道之前下载的程序是哪个逻辑块（即逻辑块序号），并根据逻辑块的序号将它存储。在追踪指纹信息时，诊断仪将发报文“$22 $F1 $5B”，ECU 将通过“$62 $F1 $5B…”，返回每一个逻辑块的指纹信息。具体格式，请参看章节 3.4.2。
（e）例程控制——“擦除内存”$31 $01 $FF $00：为了允许应用软件和数据下载，ECU 的内存将被擦除。此步骤通过例程控制服务（$31）来执行擦除内存。如果擦除内存例程被调用执行，那么应用软件的标志位将被置为无效。
（f）下载过程$34，$36，$37：应用软件或者数据的每一个连续的数据块（也叫段，可能是一个完整的应用软件或者数据，也可能是应用软件或者数据的一部分）下载到 ECU 非易失性内存中，都是遵循下面的服务顺序完成数传输：

- 请求下载（$34）；
- 传输数据（$36）；
- 请求传输退出（$37）。

单个应用软件或数据块可能需要多个数据传输（$36）请求报文来完成传输（当数据块长度超出网络层缓存大小时，就会出现这种情况）。
（g）例程控制——“检查编程完整性”$31 $01 $02 $02：此例程用来检查逻辑块的完整性。

（h）例程控制——“检查编程依赖性”$31 $01 $FF $01：一旦完成所有的应用软件或数据块/模块的下载，诊断仪将开始一个例程来触发 ECU 检查重编程的依赖性。ECU 供应商定义检查内容，但必须确保所有逻辑块的兼容性和一致性。

（i）电控单元复位$11 $01：诊断仪使用物理寻址，发送一个复位类型为硬复位的 ECU 复位（$11）服务请求报文到 CAN 网络上。通过 ECU 复位服务请求将使 ECU 结束重编程过程，返回到正常的操作模式。内存驱动代码必须从 RAM 缓存中完全清除，避免意外激活这些可能会进行非预期的内存擦除或程序操作的代码。

2.3.3 后编程步骤

（a）诊断会话控制$10 $03：诊断仪发送一个会话类型为扩展会话的诊断会话控 制服务请求报文到 CAN 网络上，使所有 ECU（包括刷写执行完毕已复位的 ECU）切换至扩展会话模式。此请求使用一个单帧请求报文，通过功能寻址发送给所有的 ECU。

（b）通信控制$28 $00 $03：诊断仪通过通信控制（$28）服务请求，开启非诊 断报文的发送和接收。请求中的控制类型参数置为“enable the transmission and the reception ”，通信类型置为“ application and network management messages”。此请求使用一个单帧请求报文，通过功能寻址发送给所有的 ECU。

（c）控制 DTC 设置 $85 $01：诊断仪发送 DTC 设置类型设为“开启”的控制 DTC 设置服务请求。此请求使用一个单帧请求报文，通过功能寻址发送给所有的 ECU。

（d）诊断会话控制$10 $01：诊断仪发送一个会话类型为默认会话的诊断会话控制 （$10） 服务请求报文到 CAN 网络上。所有的 ECU 接收到诊断会话控制（$10），而进入到默认会话模式。此请求通过功能寻址发送，发送给所有 ECU。

诊断仪应停止发送周期性的诊断仪在线请求 （$3E $80）。

（e）清除诊断信息$14 $FF $FF $FF：如果重编程 ECU 在编程步骤被重启，当编程 ECU 运行在默认会话模式时，网络上其它的 ECU 仍然在不能正常通信状态，此时，一些可能被存储在编程 ECU 中的诊断信息就应该通过物理寻址的清除诊断信息（$14）服务来清除。